

Games

Outline

I. Game as adversarial search

II. The minimax algorithm

I. Games

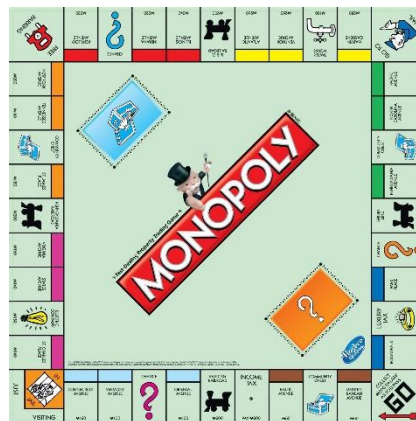
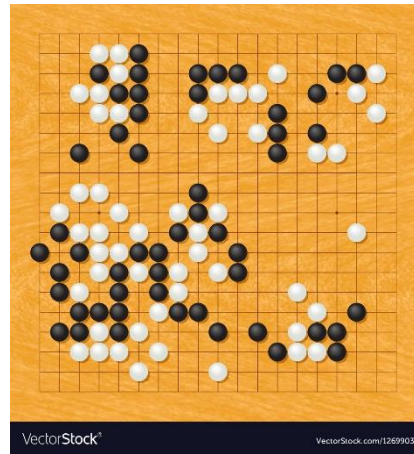
Competitive environments: goals are in conflict.

Adversarial search problems (games)

I. Games

Competitive environments: goals are in conflict.

Adversarial search problems (games)



Why Study Games?

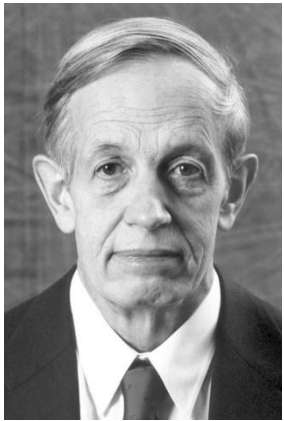
- Multiagent environment
 - ◆ Aggregate of a large number of agents for predictions (e.g., price rise).
 - ◆ Nondeterminism made by adversarial agents.
 - ◆ Introduction of new modeling techniques.

Why Study Games?

- Multiagent environment
 - ◆ Aggregate of a large number of agents for predictions (e.g., price rise).
 - ◆ Nondeterminism made by adversarial agents.
 - ◆ Introduction of new modeling techniques.
- Mathematical game theory – an important branch of economics.

Why Study Games?

- Multiagent environment
 - ◆ Aggregate of a large number of agents for predictions (e.g., price rise).
 - ◆ Nondeterminism made by adversarial agents.
 - ◆ Introduction of new modeling techniques.
- Mathematical game theory – an important branch of economics.



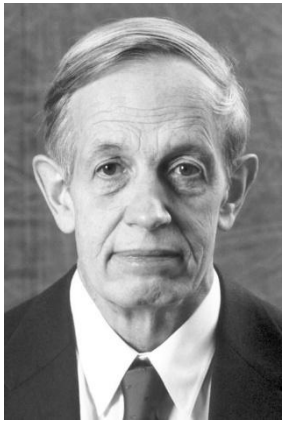
John Nash (Princeton)
Nobel Prize in Economics (1994)

Nash equilibrium for a non-cooperative game:

Each player has chosen a strategy and no player can increase own expected payoff by changing their strategy while the other players keep theirs unchanged,.

Why Study Games?

- Multiagent environment
 - ◆ Aggregate of a large number of agents for predictions (e.g., price rise).
 - ◆ Nondeterminism made by adversarial agents.
 - ◆ Introduction of new modeling techniques.
- Mathematical game theory – an important branch of economics.



John Nash (Princeton)
Nobel Prize in Economics (1994)

Nash equilibrium for a non-cooperative game:

Each player has chosen a strategy and no player can increase own expected payoff by changing their strategy while the other players keep theirs unchanged,.

- Appealing subject for study in AI.
 - ◆ Fun and entertaining.
 - ◆ Hard – engaging the intellectual faculties of humans.
 - ◆ Abstract nature – easy to represent with small number of actions.

* Photo from <https://www.nobelprize.org/prizes/economic-sciences/1994/nash/facts/>.

History of Computer Games



Claude Shannon (MIT)
“Father of information theory”
National Medal of Science (1966)

- 1950 Claude Shannon, *Programming a Computer for Playing Chess*.
- 1956 John McCarthy conceives alpha-beta search.
- 1982 BELLE becomes the first chess program to achieve master status.
- 1984 Judea Pearl, *Heuristics*.
- 1997 Deep Blue (IBM) defeats world chess champion Garry Kasparov.
- 2017 AlphaGo (Alphabet) defeats world's no. 1 Go player Ke Jie.
 - Visual pattern recognition
 - Reinforcement learning
 - Neural networks
 - Monte Carlo tree search
- 2018 AlphaZero (Alphabet) defeats top programs in Go, chess, shogi.
- 2019 Pluribus (CMU) defeats top-ranked players in Texas hold'em games with six players.

* Photo from https://en.wikipedia.org/wiki/Claude_Shannon.

Types of Games

- ♣ Games with deterministic, perfect information (e.g., chess, go, checkers)
- ♣ Stochastic games (e.g., backgammon)
 - Probabilistic transitions by players
- ♣ Partially observable games (e.g., bridge, poker)

II. Two-Player Game

- ♦ Perfect information – fully observable.
- ♦ **Zero sum** – what is good for one player is just as bad for the other.

move $\Leftrightarrow$ action
position $\Leftrightarrow$ state

MAX and MIN: two players.

Formal Definition of a Game

- s_0 : initial state – game setup.

Formal Definition of a Game

- s_0 : initial state – game setup.

At a state s :

- $\text{TO-MOVE}(s)$: the player to move in the state.

Formal Definition of a Game

- s_0 : initial state – game setup.

At a state s :

- $\text{TO-MOVE}(s)$: the player to move in the state.
- $\text{ACTIONS}(s)$: the set of legal moves in the state.

Formal Definition of a Game

- s_0 : initial state – game setup.

At a state s :

- $\text{TO-MOVE}(s)$: the player to move in the state.
- $\text{ACTIONS}(s)$: the set of legal moves in the state.
- $\text{RESULT}(s, a)$: the transition model defining the next state from taking action a .

Formal Definition of a Game

- s_0 : initial state – game setup.

At a state s :

- $\text{TO-MOVE}(s)$: the player to move in the state.
- $\text{ACTIONS}(s)$: the set of legal moves in the state.
- $\text{RESULT}(s, a)$: the transition model defining the next state from taking action a .
- $\text{IS-TERMINAL}(s)$: to test if the game is over, i.e., if s is a terminal state.

Formal Definition of a Game

- s_0 : initial state – game setup.

At a state s :

- $\text{TO-MOVE}(s)$: the player to move in the state.
- $\text{ACTIONS}(s)$: the set of legal moves in the state.
- $\text{RESULT}(s, a)$: the transition model defining the next state from taking action a .
- $\text{IS-TERMINAL}(s)$: to test if the game is over, i.e., if s is a terminal state.
- $\text{UTILITY}(s, p)$: a **utility function** to return a value to the player p if the game ends in terminal state s .

Formal Definition of a Game

- s_0 : initial state – game setup.

At a state s :

- $\text{TO-MOVE}(s)$: the player to move in the state.
- $\text{ACTIONS}(s)$: the set of legal moves in the state.
- $\text{RESULT}(s, a)$: the transition model defining the next state from taking action a .
- $\text{IS-TERMINAL}(s)$: to test if the game is over, i.e., if s is a terminal state.
- $\text{UTILITY}(s, p)$: a **utility function** to return a value to the player p if the game ends in terminal state s .

e.g., in chess, win (1), loss (0), draw (1/2)

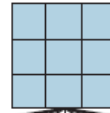
Total payoff for all players is constant (**zero-sum game**):

$$1 + 0 = 0 + 1 = \frac{1}{2} + \frac{1}{2} = 1$$

State Space Graph (Tic-Tac-Toe)

Vertices $\leftrightarrow$ states and edges $\leftrightarrow$ moves

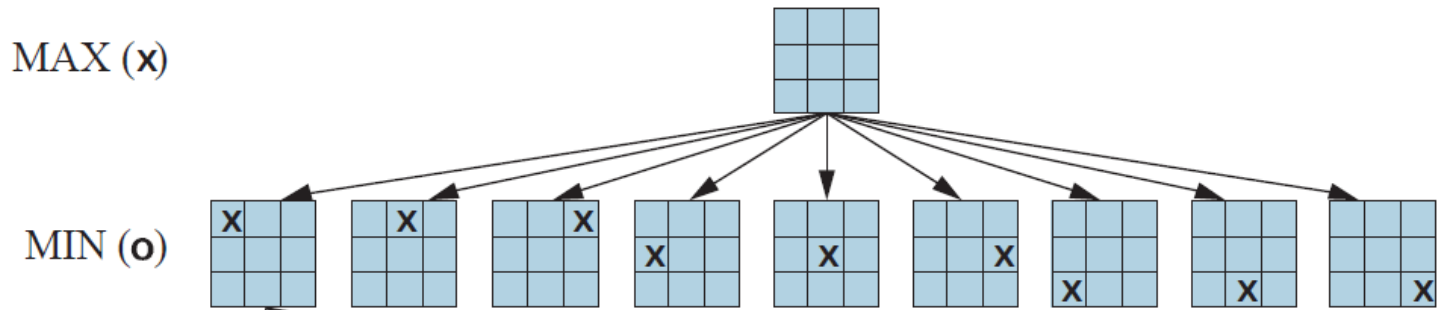
MAX (x)



- - - -

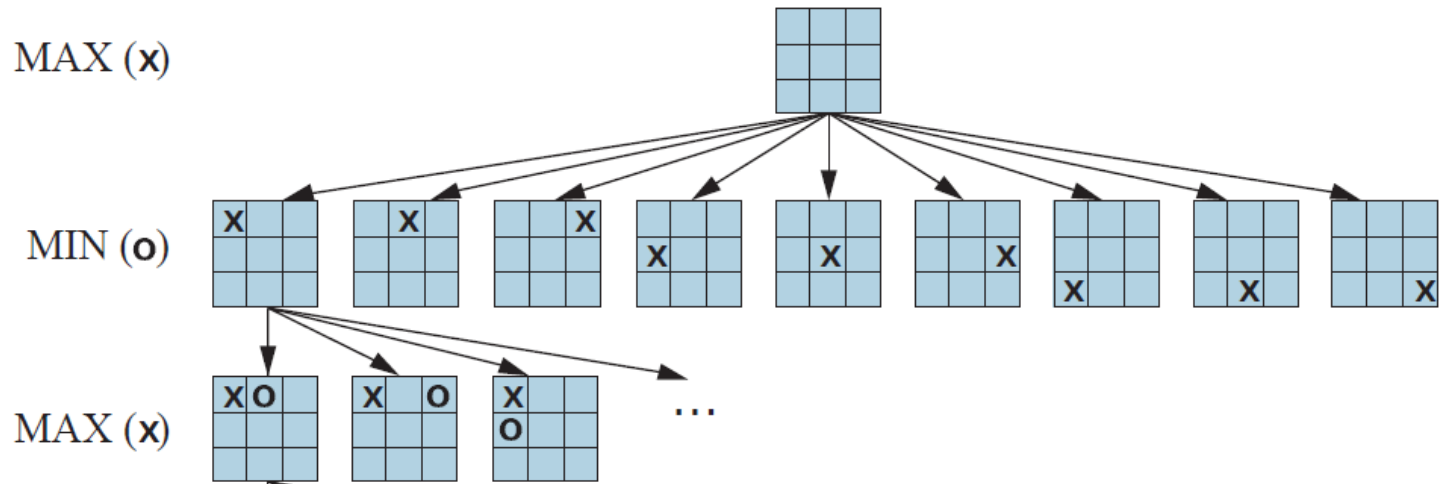
State Space Graph (Tic-Tac-Toe)

Vertices $\leftrightarrow$ states and edges $\leftrightarrow$ moves



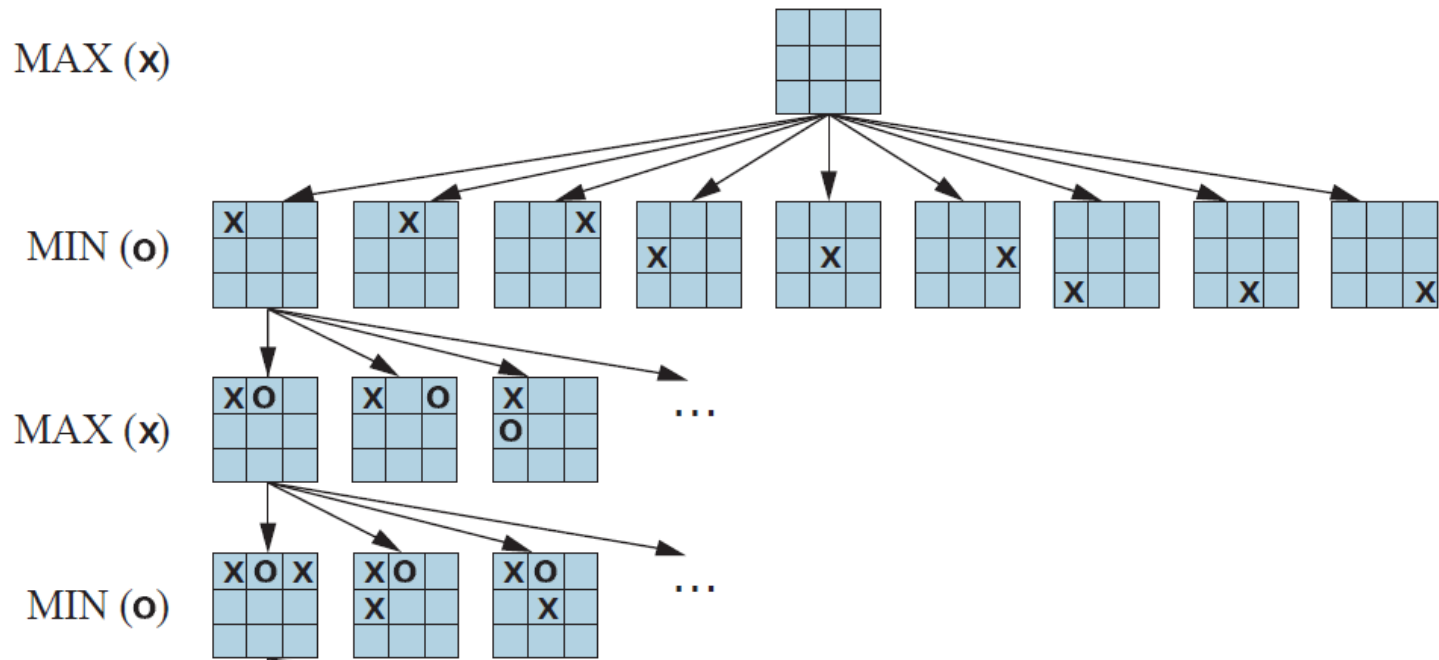
State Space Graph (Tic-Tac-Toe)

Vertices $\leftrightarrow$ states and edges $\leftrightarrow$ moves



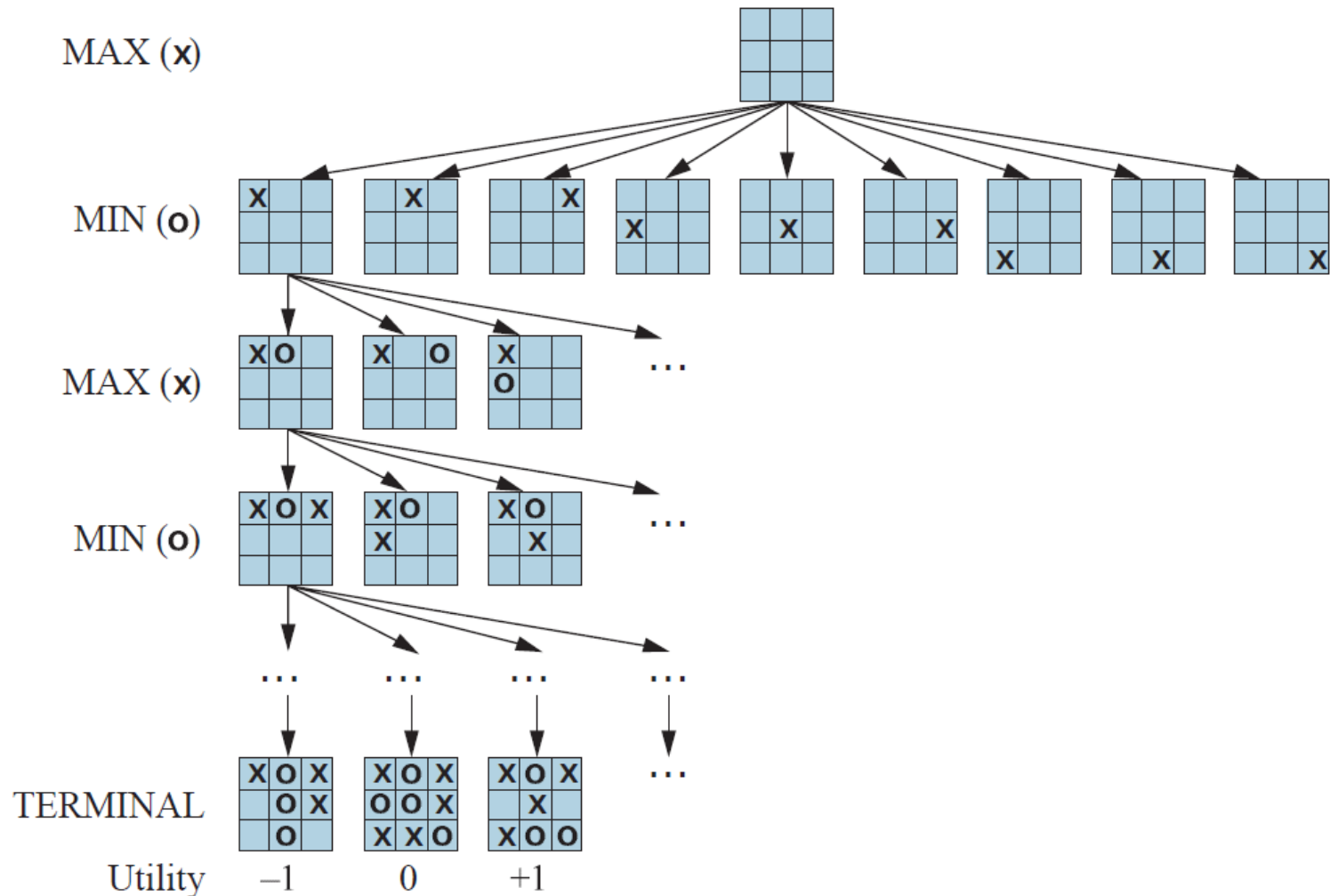
State Space Graph (Tic-Tac-Toe)

Vertices $\leftrightarrow$ states and edges $\leftrightarrow$ moves



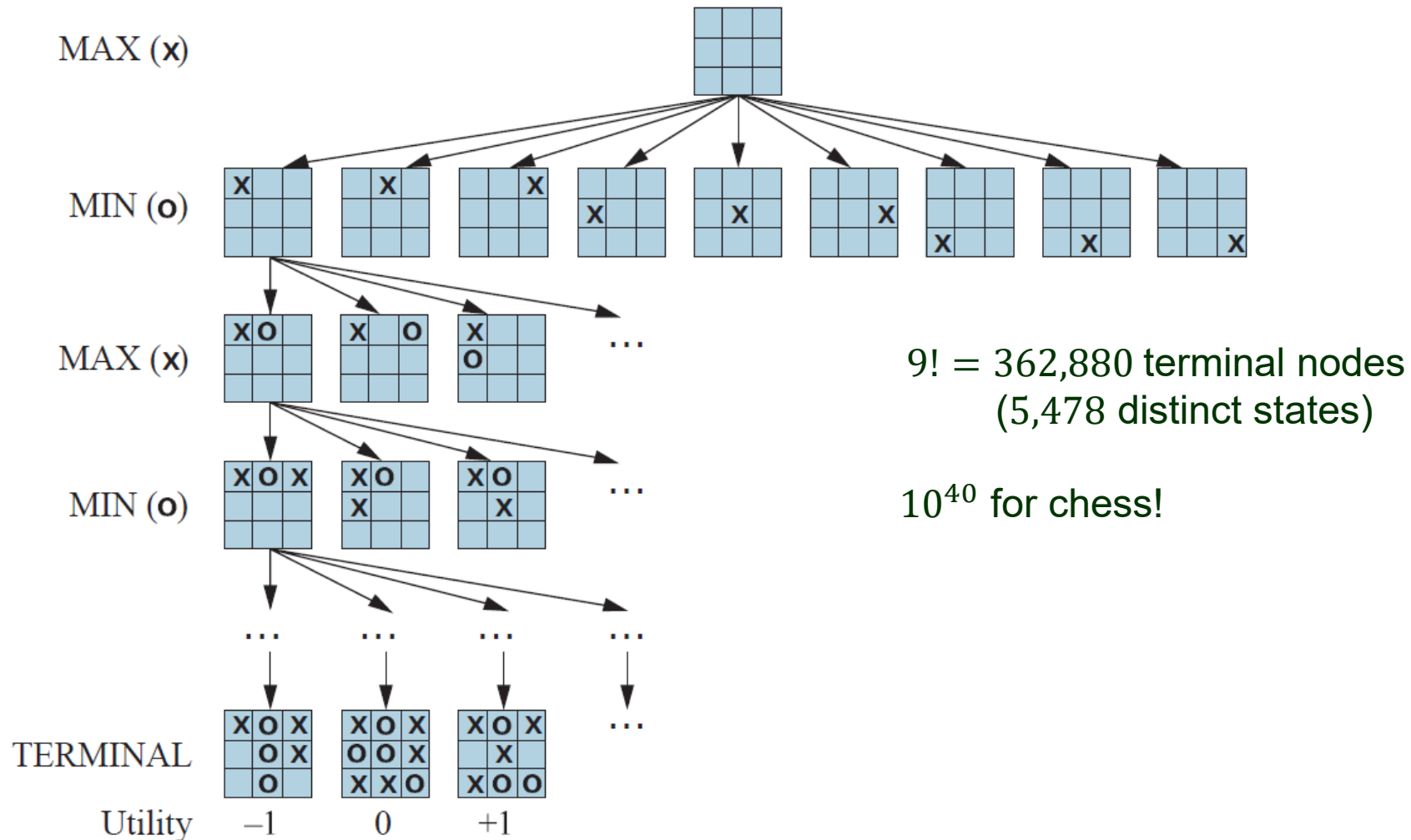
State Space Graph (Tic-Tac-Toe)

Vertices $\leftrightarrow$ states and edges $\leftrightarrow$ moves



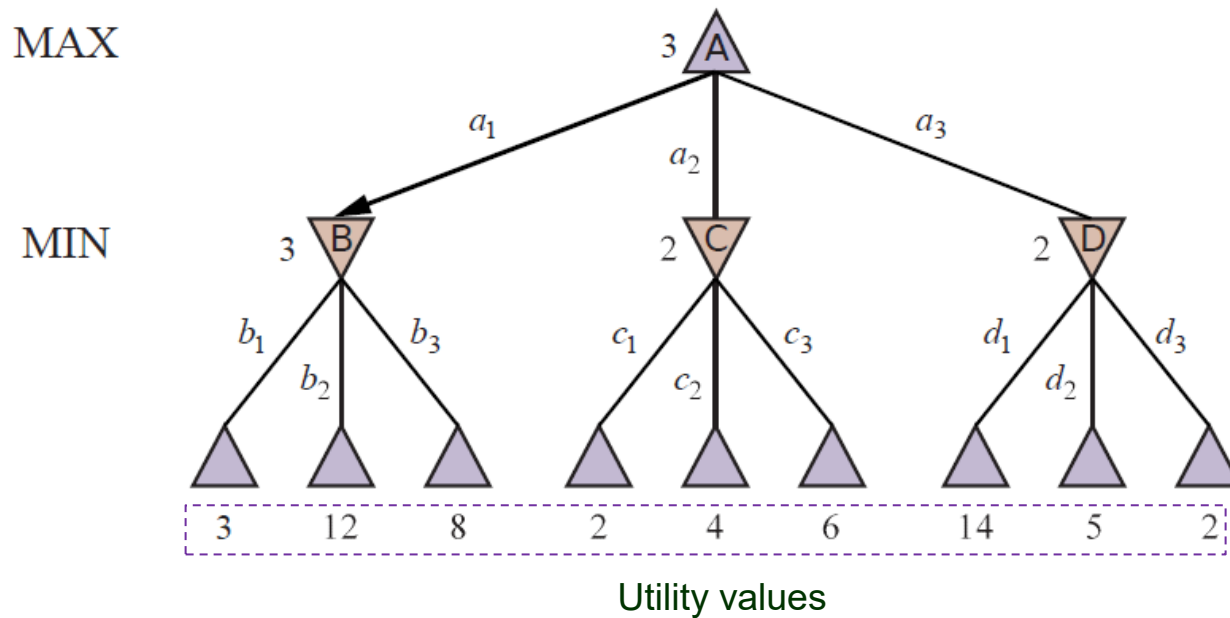
State Space Graph (Tic-Tac-Toe)

Vertices $\leftrightarrow$ states and edges $\leftrightarrow$ moves



Two-Ply Game Tree

Ply: one move by a player



Optimal Strategy

Work out the minimax value of every state s in the tree,

$$\text{MINIMAX}(s)$$

assuming both players play optimally:

- MAX moves to a state of maximum value at its turn;
- MIN moves to a state of minimum value at its turn.

Optimal Strategy

Work out the minimax value of every state s in the tree,

$$\text{MINIMAX}(s)$$

assuming both players play optimally:

- MAX moves to a state of maximum value at its turn;
- MIN moves to a state of minimum value at its turn.

$$\text{MINIMAX}(s) =$$

Optimal Strategy

Work out the minimax value of every state s in the tree,

$\text{MINIMAX}(s)$

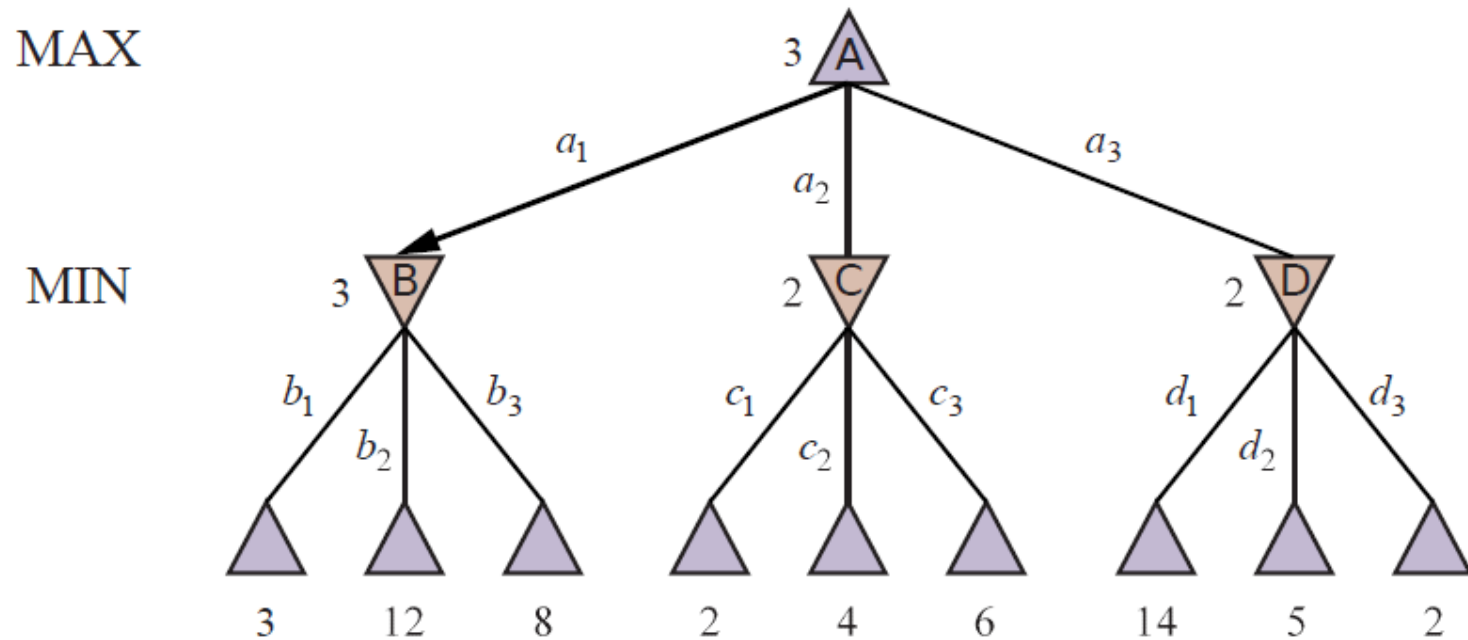
assuming both players play optimally:

- MAX moves to a state of maximum value at its turn;
- MIN moves to a state of minimum value at its turn.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

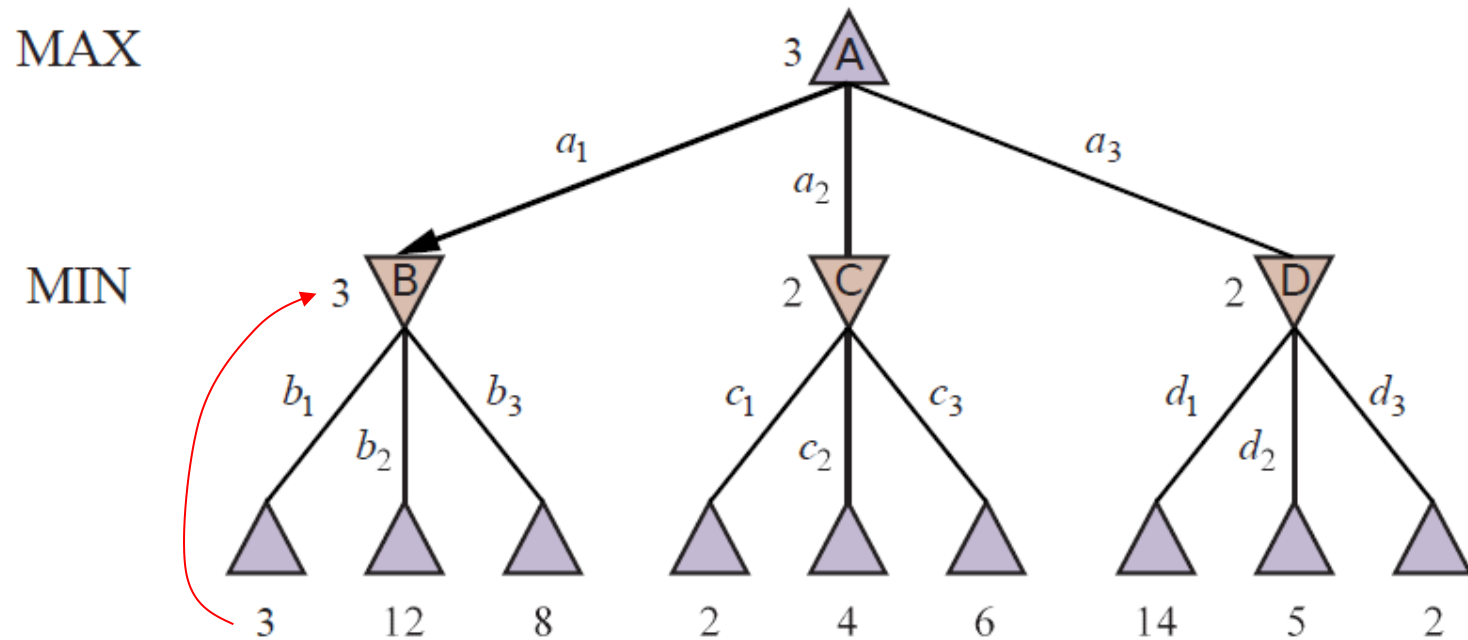
Minimax Value at Min Nodes

MIN: choose a move to a MAX node with the lowest value.



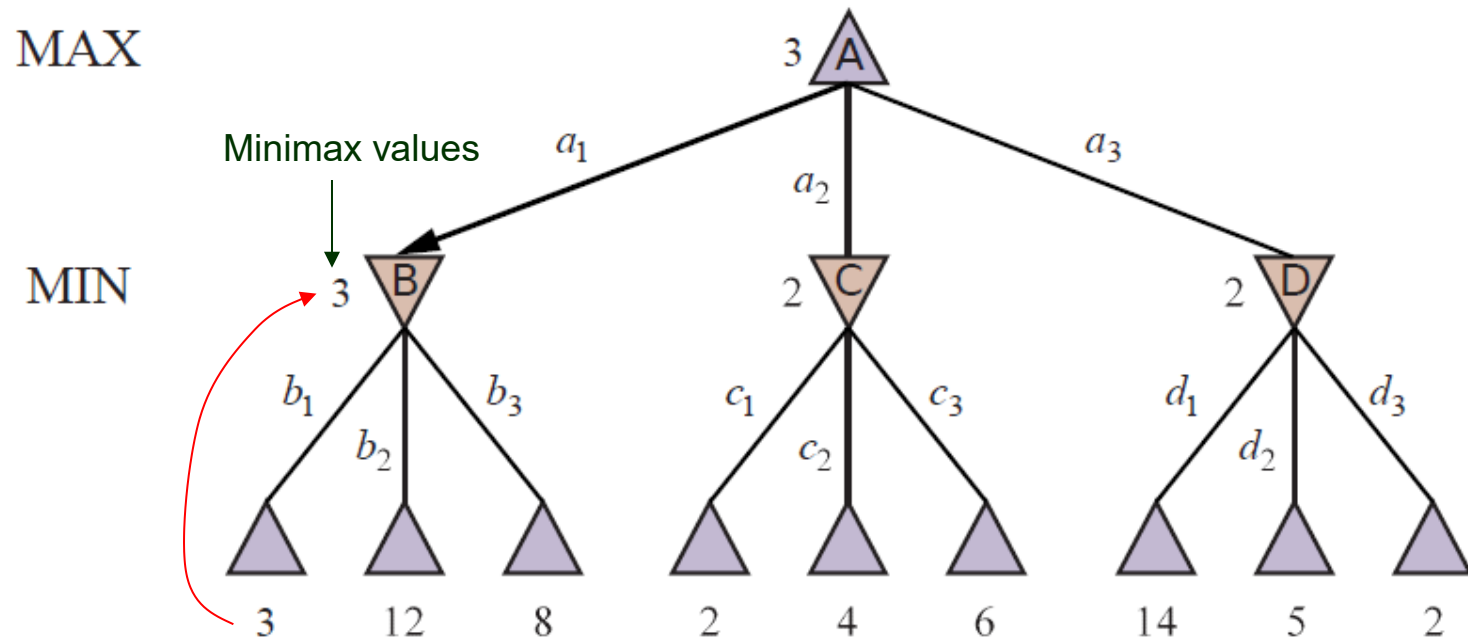
Minimax Value at Min Nodes

MIN: choose a move to a MAX node with the lowest value.



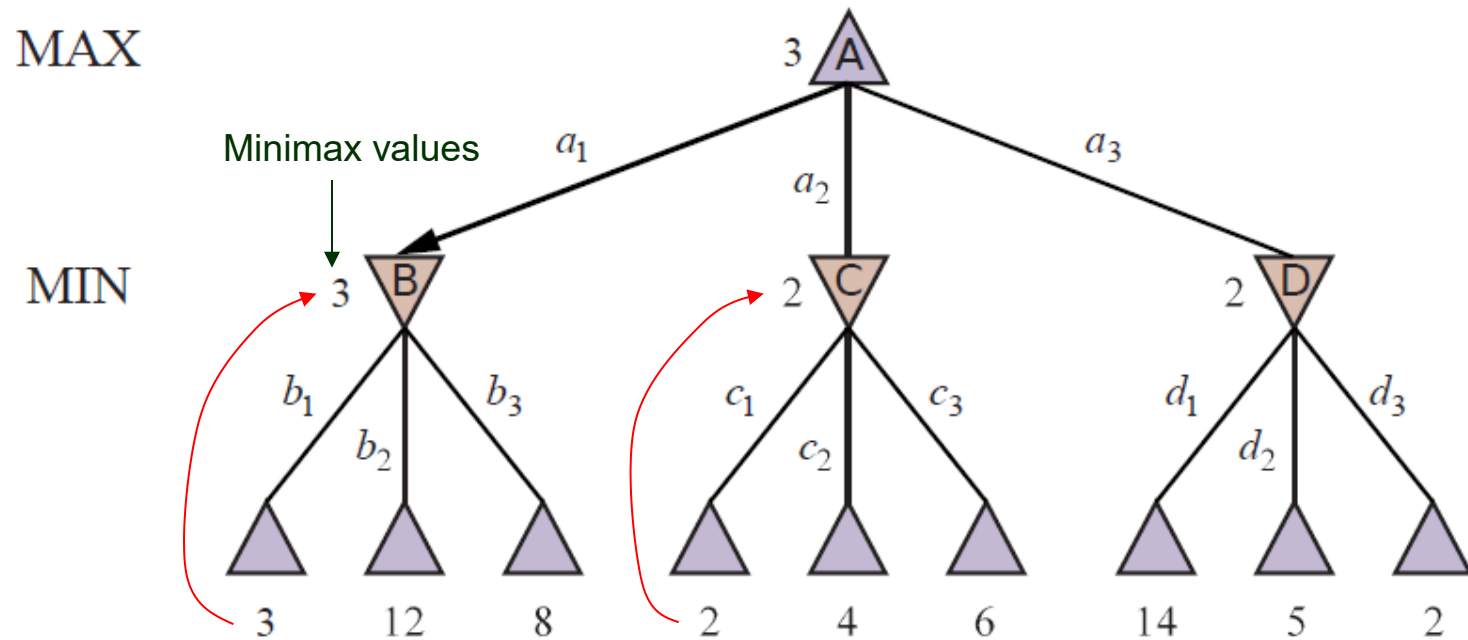
Minimax Value at Min Nodes

MIN: choose a move to a MAX node with the lowest value.



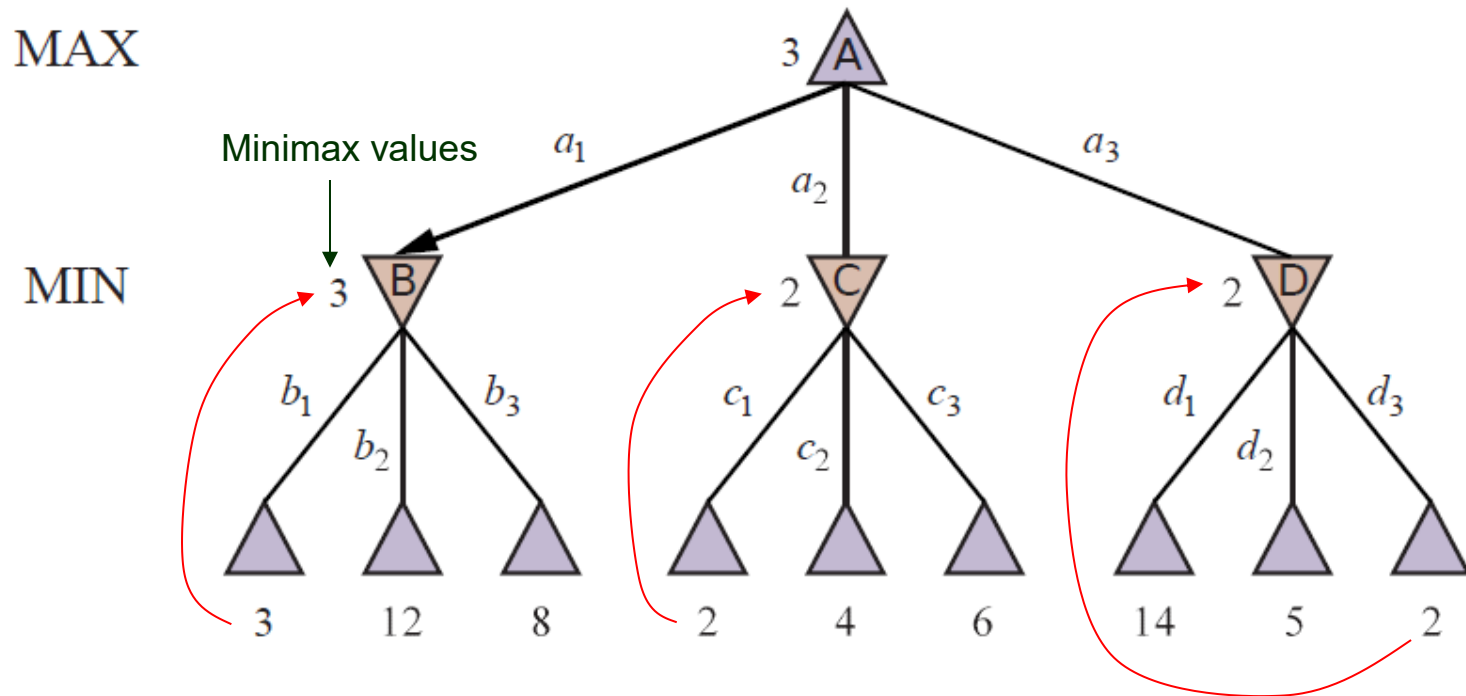
Minimax Value at Min Nodes

MIN: choose a move to a MAX node with the lowest value.



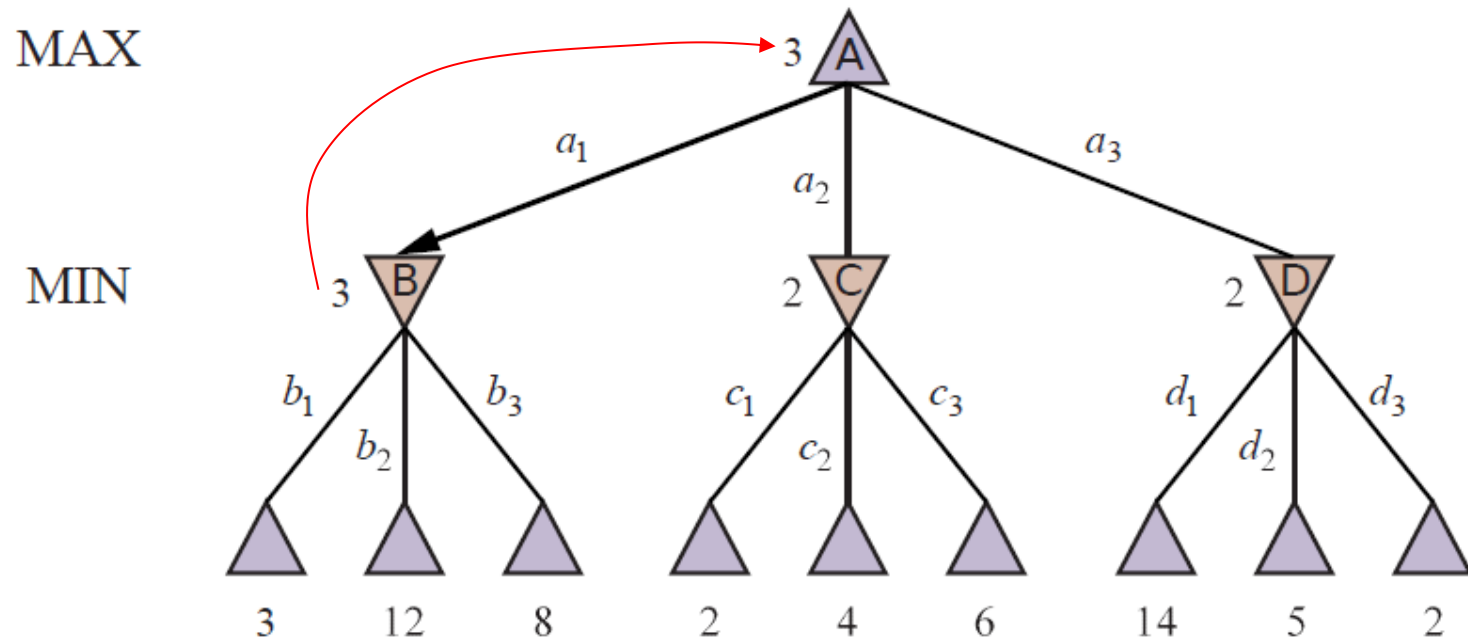
Minimax Value at Min Nodes

MIN: choose a move to a MAX node with the lowest value.

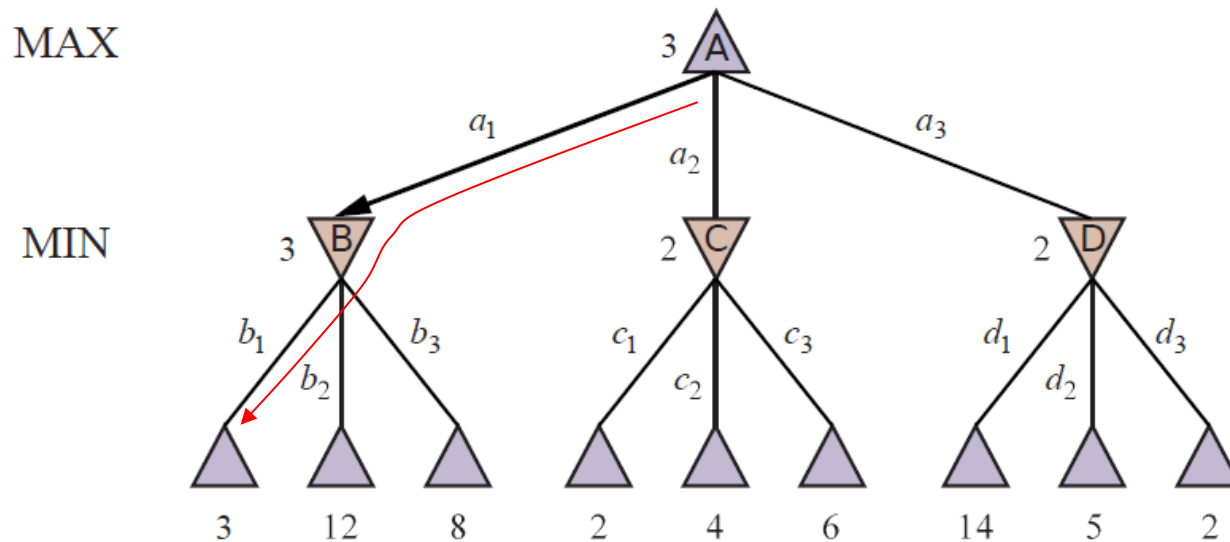


Minimax Value at a Max Node

MAX: choose a move to a MIN node with the highest value.



Solution of the Game



Best move for MAX: a_1

Best move for MIN in response: b_1

The Minimax Search Algorithm

function MINIMAX-SEARCH(*game, state*) **returns** *an action*

$\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$

if $\text{player} = \text{MAX}$ **then** $\text{value, move} \leftarrow \text{MAX-VALUE}(\text{game, state})$
 else $\text{value, move} \leftarrow \text{MIN-VALUE}(\text{game, state})$

return *move*

function MAX-VALUE(*game, state*) **returns** *a (utility, move) pair*

if $\text{game.IS-TERMINAL}(\text{state})$ **then return** $\text{game.UTILITY}(\text{state, player}), \text{null}$

$v \leftarrow -\infty$

for each a **in** $\text{game.ACTIONS}(\text{state})$ **do**

$v2, a2 \leftarrow \text{MIN-VALUE}(\text{game, game.RESULT}(\text{state, } a))$ // $a2$ is discarded

if $v2 > v$ **then**

$v, \text{move} \leftarrow v2, a$

return v, move

function MIN-VALUE(*game, state*) **returns** *a (utility, move) pair*

if $\text{game.IS-TERMINAL}(\text{state})$ **then return** $\text{game.UTILITY}(\text{state, player}), \text{null}$

$v \leftarrow +\infty$

for each a **in** $\text{game.ACTIONS}(\text{state})$ **do**

$v2, a2 \leftarrow \text{MAX-VALUE}(\text{game, game.RESULT}(\text{state, } a))$ // $a2$ is discarded

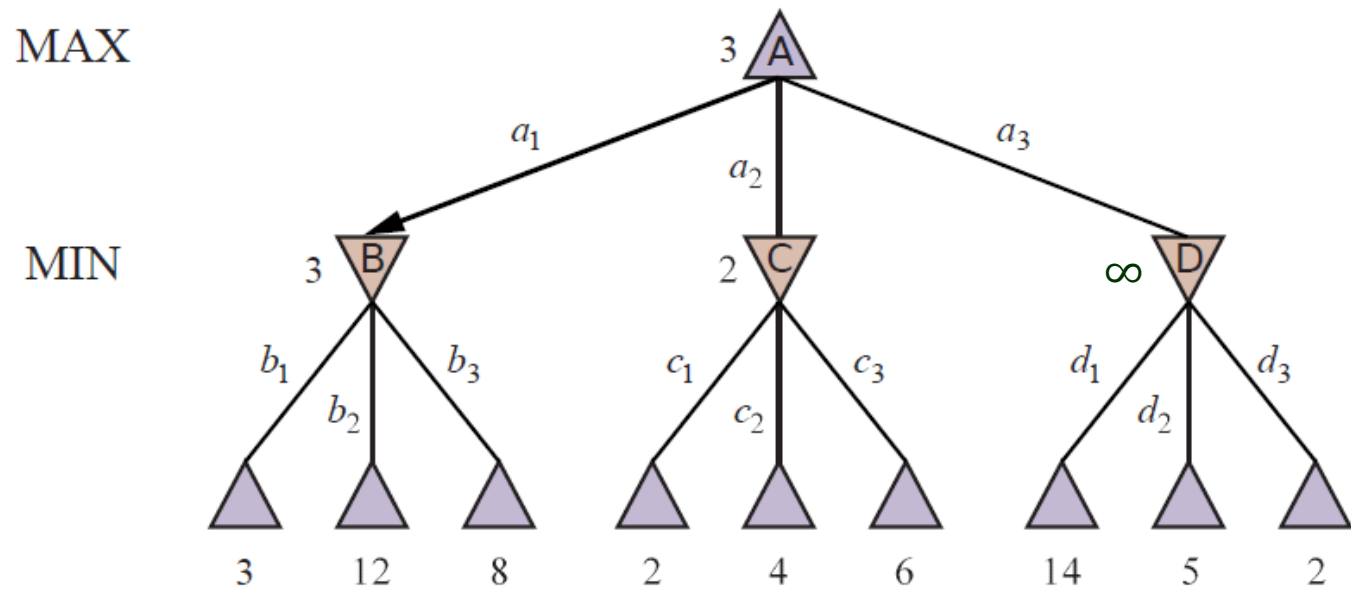
if $v2 < v$ **then**

$v, \text{move} \leftarrow v2, a$

return v, move

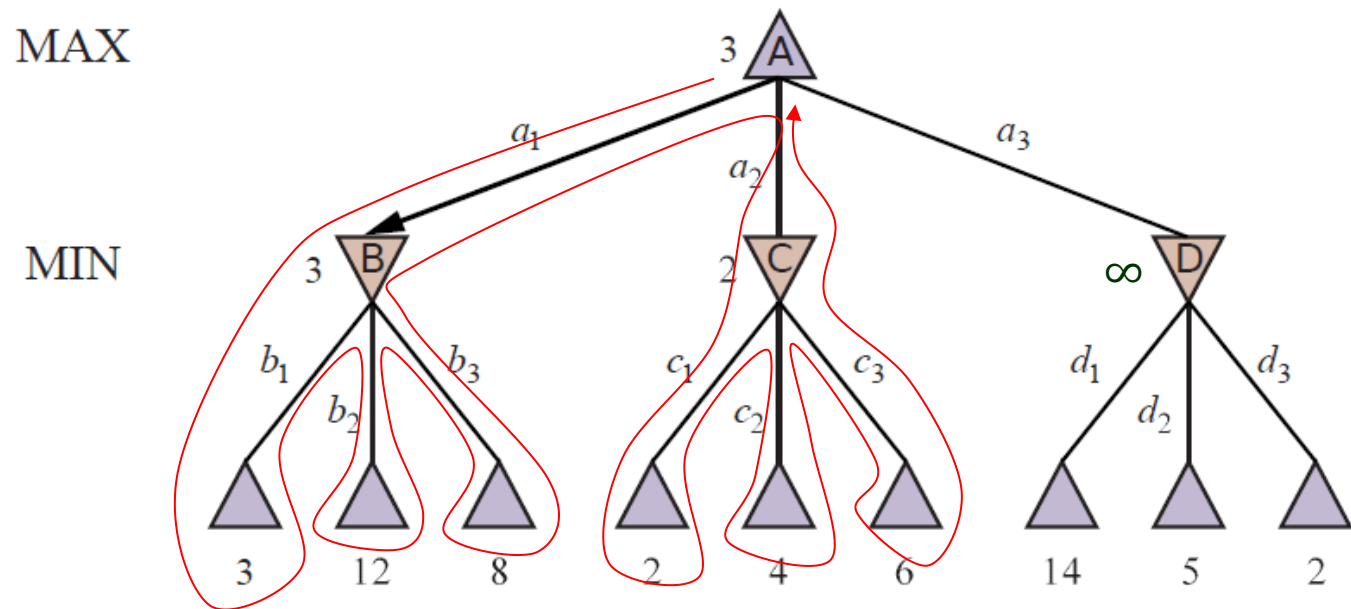
Algorithm Execution

Depth-first search with backed-up value on return from a node.



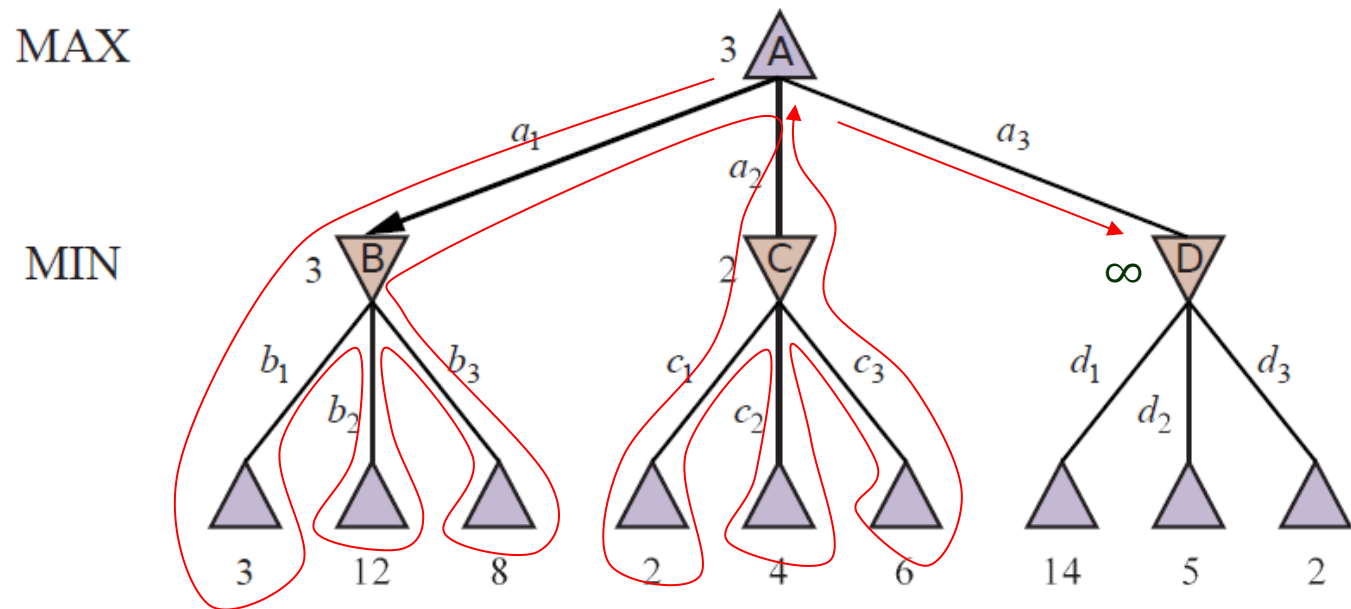
Algorithm Execution

Depth-first search with backed-up value on return from a node.

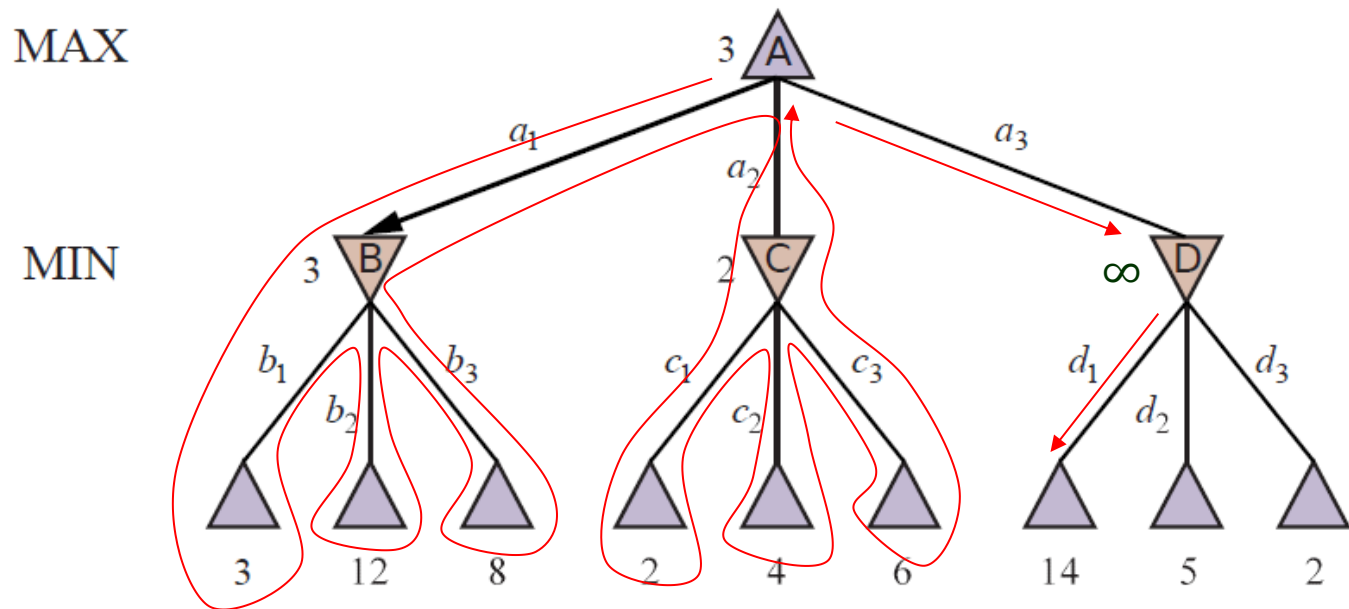


Algorithm Execution

Depth-first search with backed-up value on return from a node.

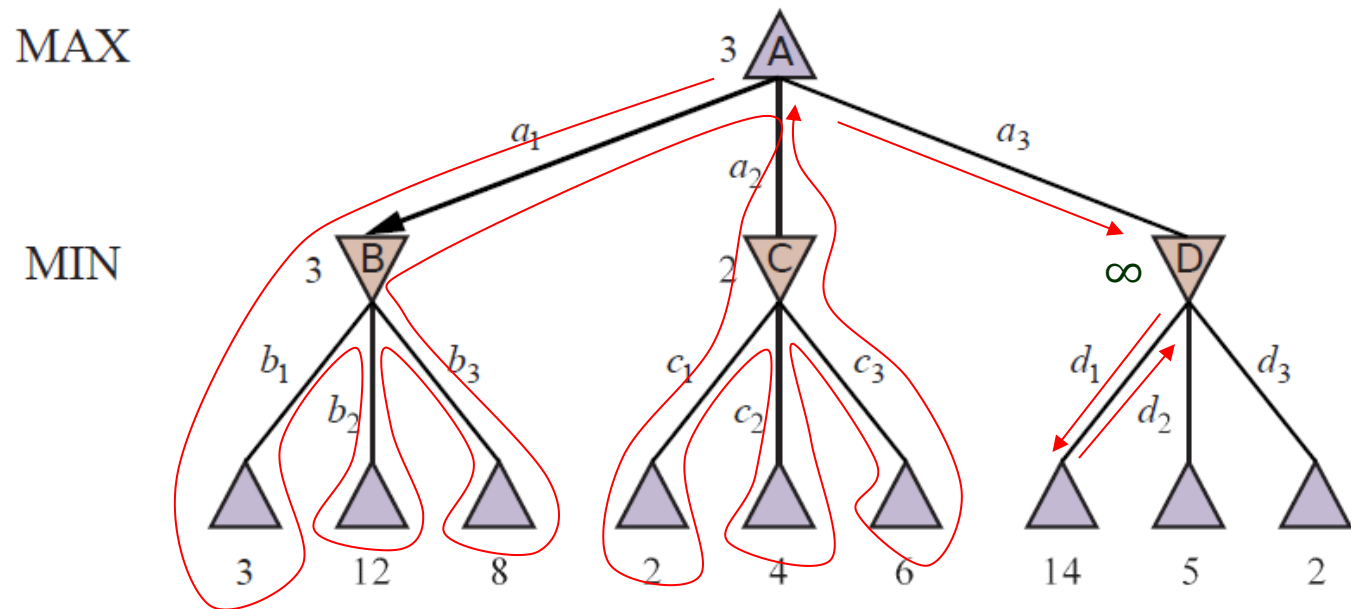


Depth-first search with backed-up value on return from a node.



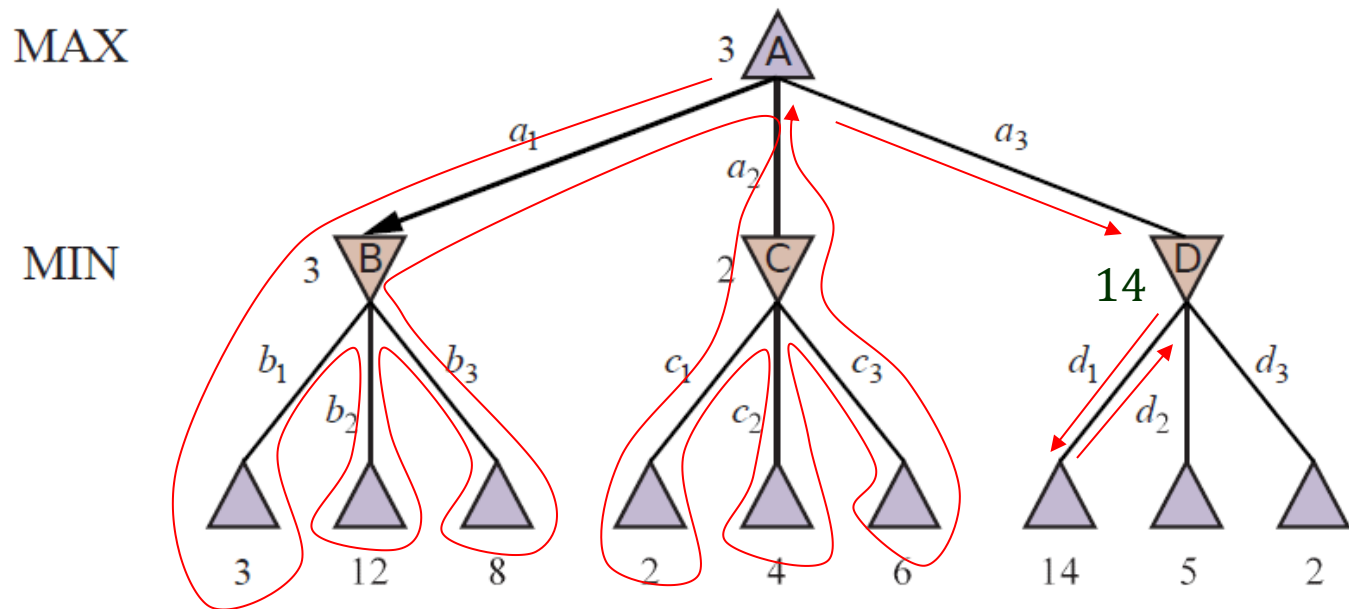
Algorithm Execution

Depth-first search with backed-up value on return from a node.



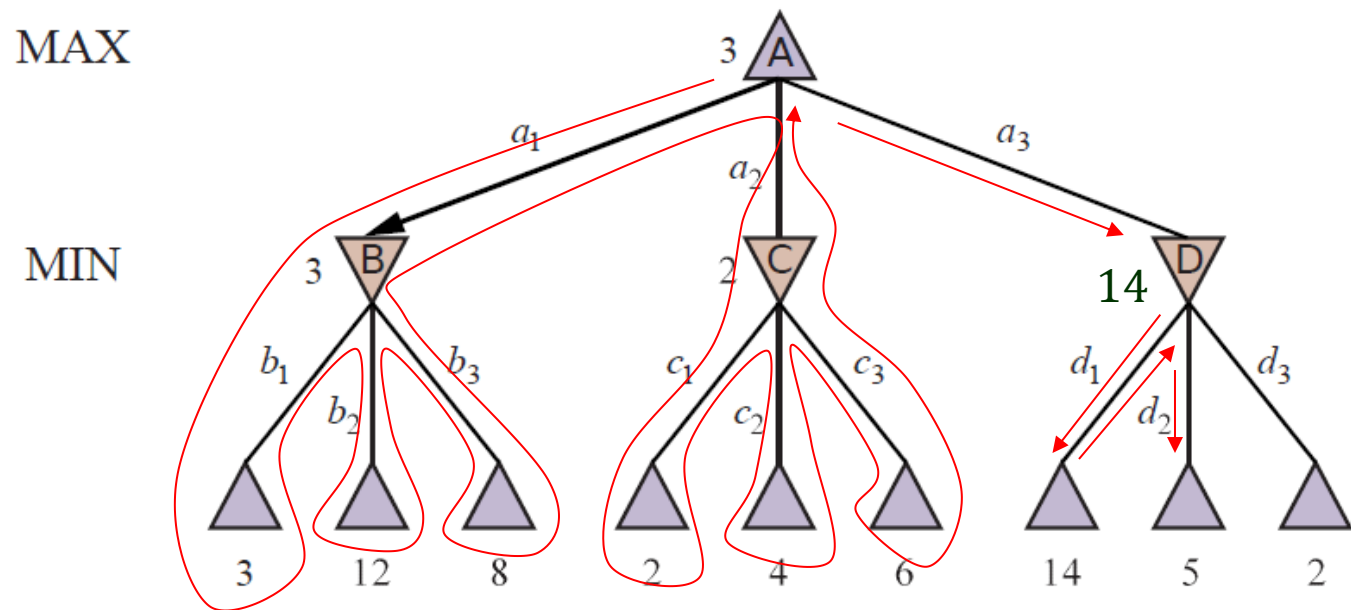
Algorithm Execution

Depth-first search with backed-up value on return from a node.



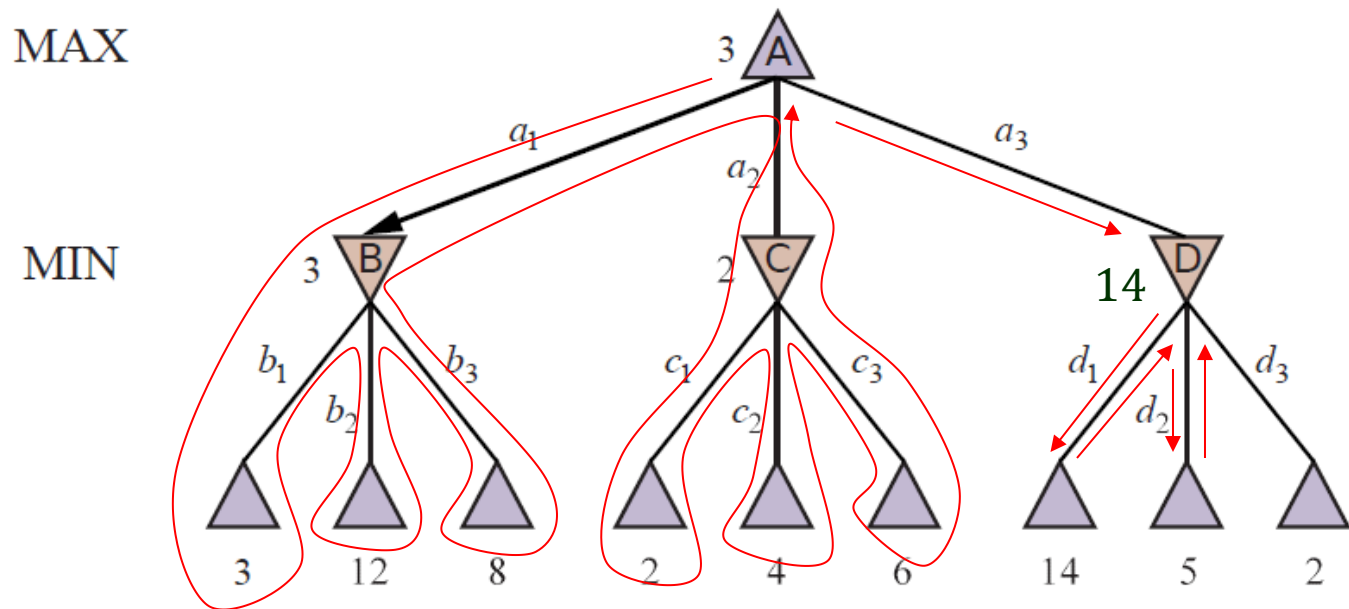
Algorithm Execution

Depth-first search with backed-up value on return from a node.



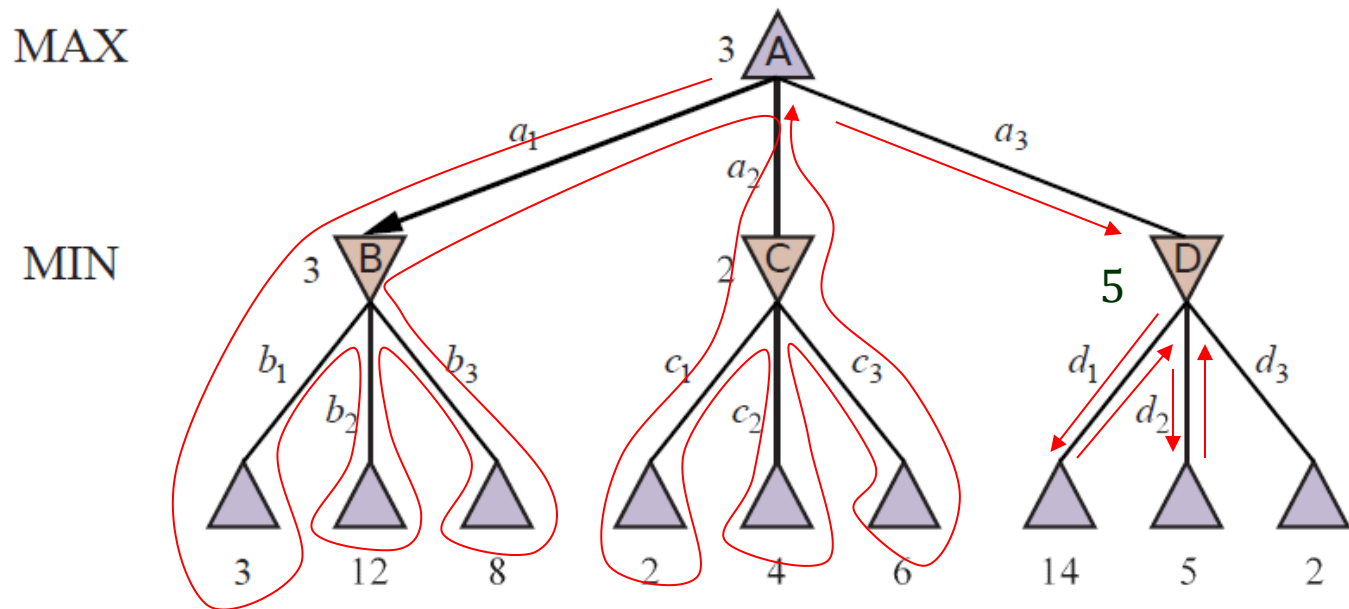
Algorithm Execution

Depth-first search with backed-up value on return from a node.



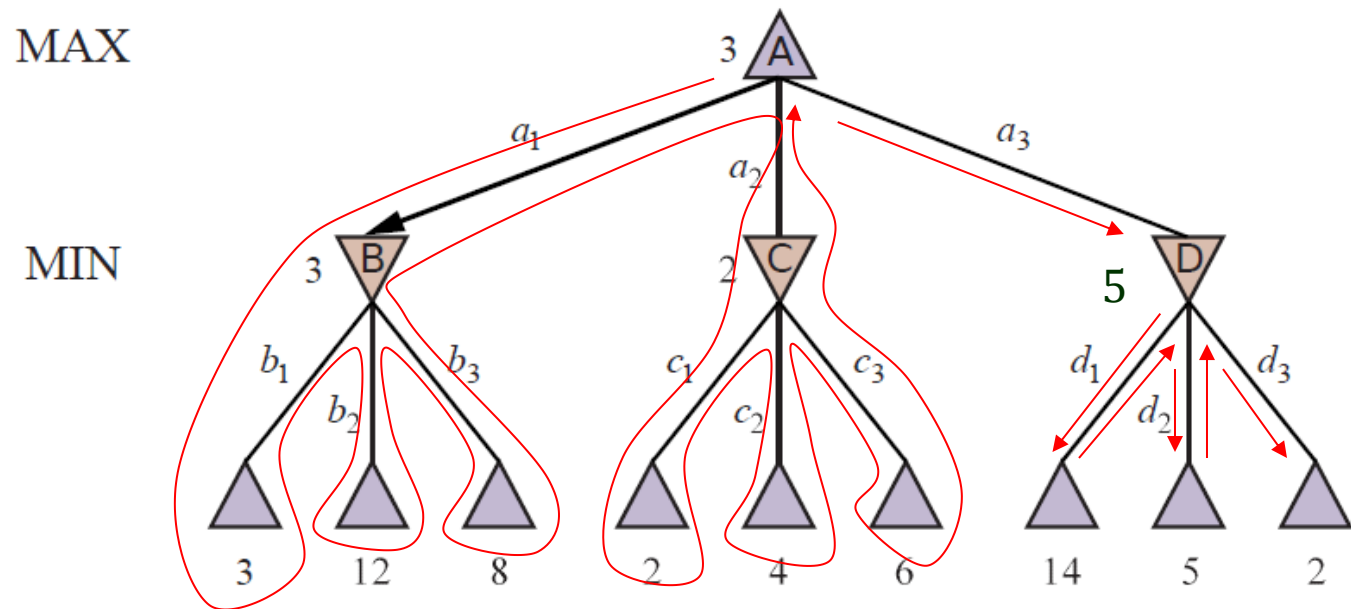
Algorithm Execution

Depth-first search with backed-up value on return from a node.



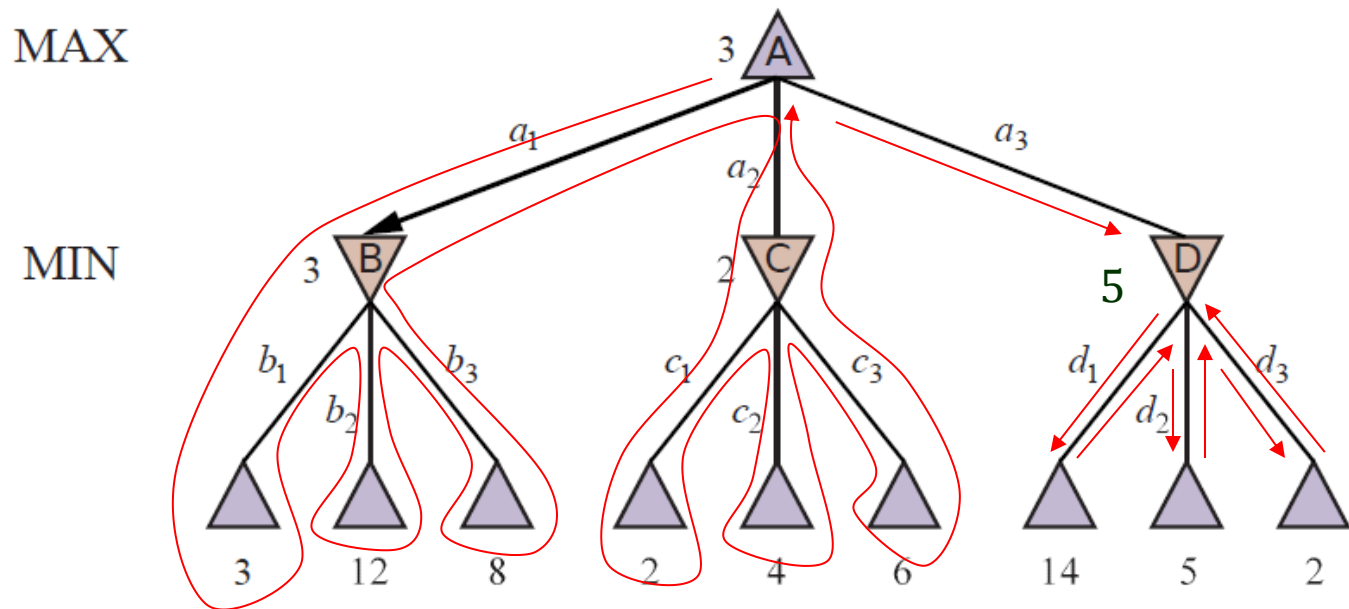
Algorithm Execution

Depth-first search with backed-up value on return from a node.



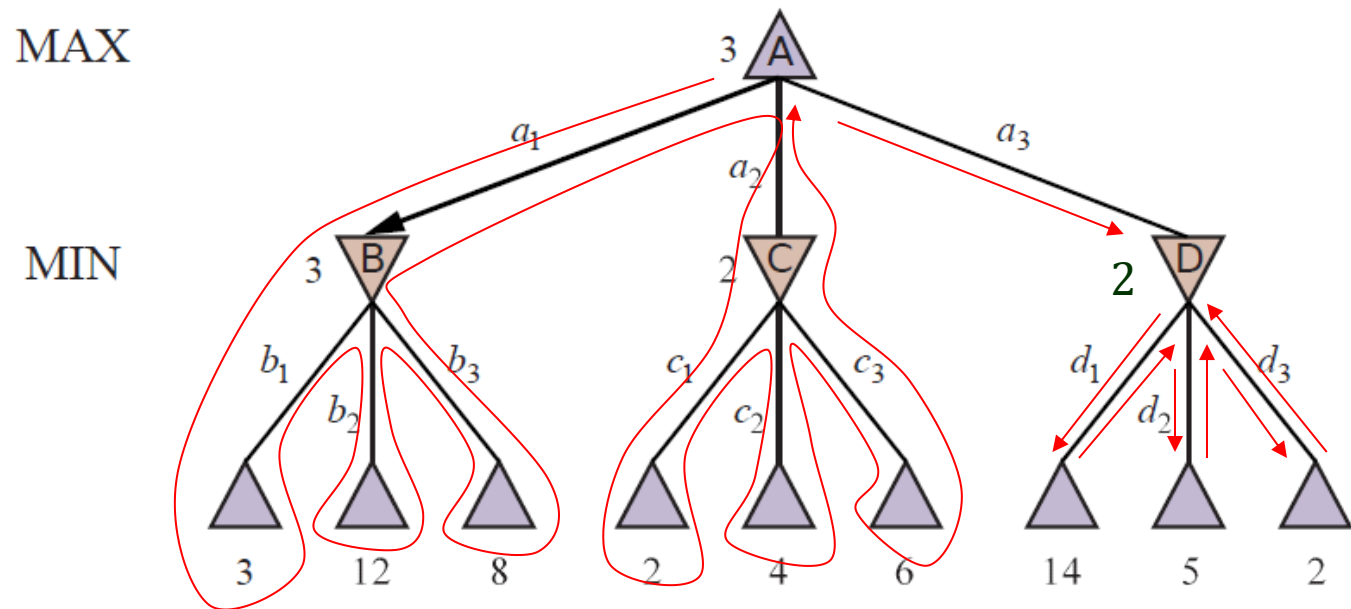
Algorithm Execution

Depth-first search with backed-up value on return from a node.



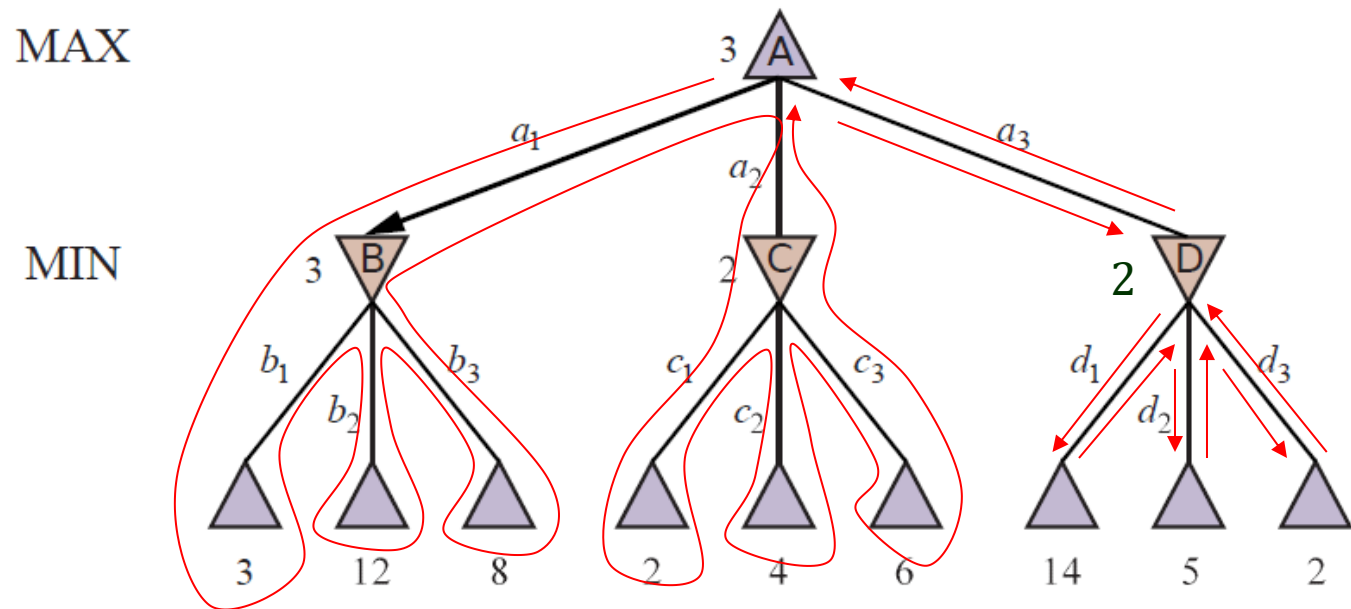
Algorithm Execution

Depth-first search with backed-up value on return from a node.



Algorithm Execution

Depth-first search with backed-up value on return from a node.



Summary on Minimax

- ♦ Complete if the game tree is finite.

Summary on Minimax

- ♦ Complete if the game tree is finite.
- ♦ Optimal against an optimal opponent.

Summary on Minimax

- ♦ Complete if the game tree is finite.
- ♦ Optimal against an optimal opponent.

If MIN does not play optimally,

- 1) MAX will play at least as well as an optimal player;
- 2) but there may be a better strategy against the suboptimal MIN.

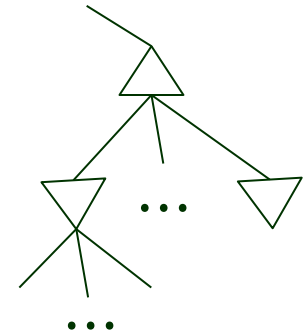
Summary on Minimax

- ♦ Complete if the game tree is finite.
- ♦ Optimal against an optimal opponent.

If MIN does not play optimally,

- 1) MAX will play at least as well as an optimal player;
- 2) but there may be a better strategy against the suboptimal MIN.

- ♦ Complexities:



Summary on Minimax

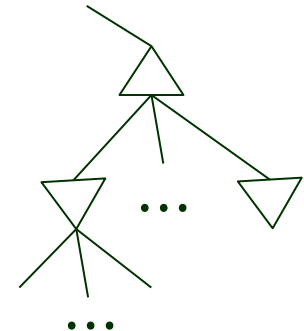
- ♦ Complete if the game tree is finite.
- ♦ Optimal against an optimal opponent.

If MIN does not play optimally,

- 1) MAX will play at least as well as an optimal player;
- 2) but there may be a better strategy against the suboptimal MIN.

- ♦ Complexities:

branching factor max depth
 ↙ ↘
Time: $O(b^m)$



Summary on Minimax

- ♦ Complete if the game tree is finite.
- ♦ Optimal against an optimal opponent.

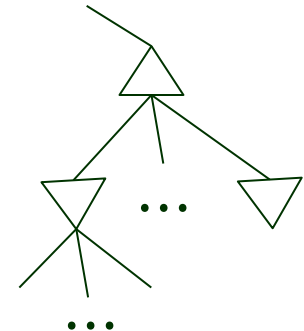
If MIN does not play optimally,

- 1) MAX will play at least as well as an optimal player;
- 2) but there may be a better strategy against the suboptimal MIN.

- ♦ Complexities:

branching factor max depth

Time: $O(b^m)$ Space: $O(bm)$



Summary on Minimax

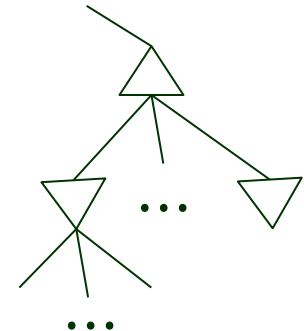
- ♦ Complete if the game tree is finite.
- ♦ Optimal against an optimal opponent.

If MIN does not play optimally,

- 1) MAX will play at least as well as an optimal player;
- 2) but there may be a better strategy against the suboptimal MIN.

- ♦ Complexities:

branching factor max depth
 ↙ ↘
Time: $O(b^m)$ Space: $O(bm)$



Chess: $b \approx 35$ and $m \approx 100$ for a reasonable game.
Exact optimal solution infeasible!

Multiplayer Games

Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C

Utility vector

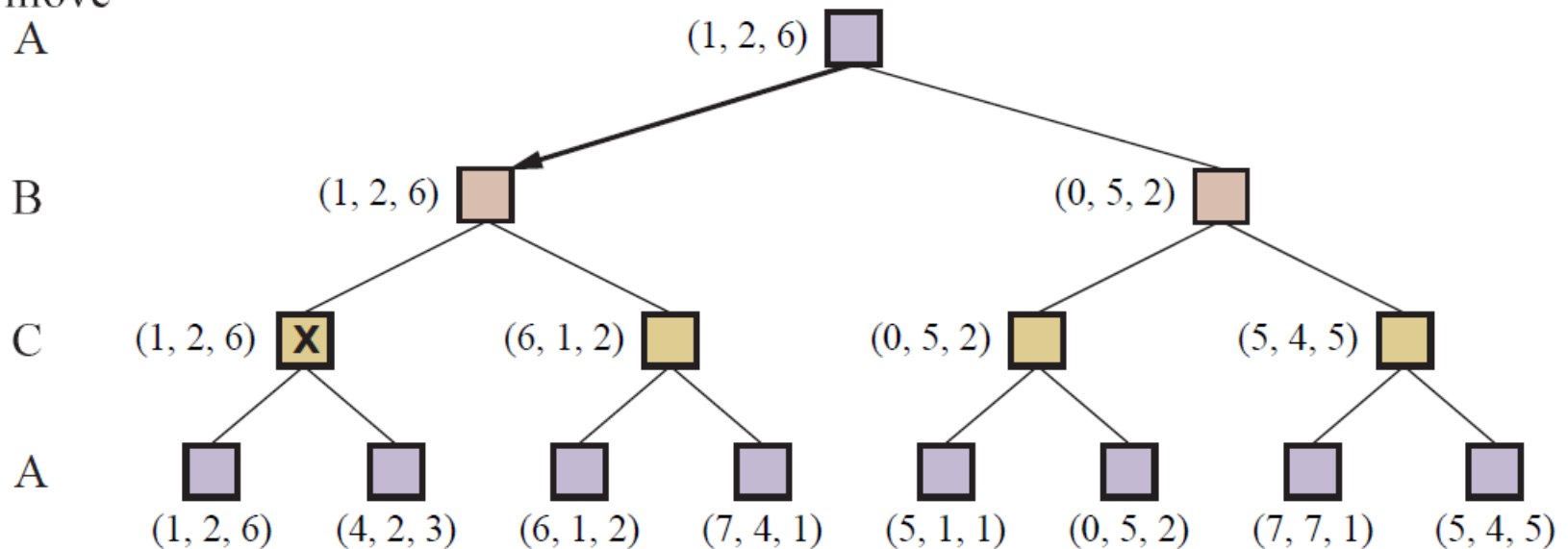
Multiplayer Games

Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C
Utility vector

to move
A



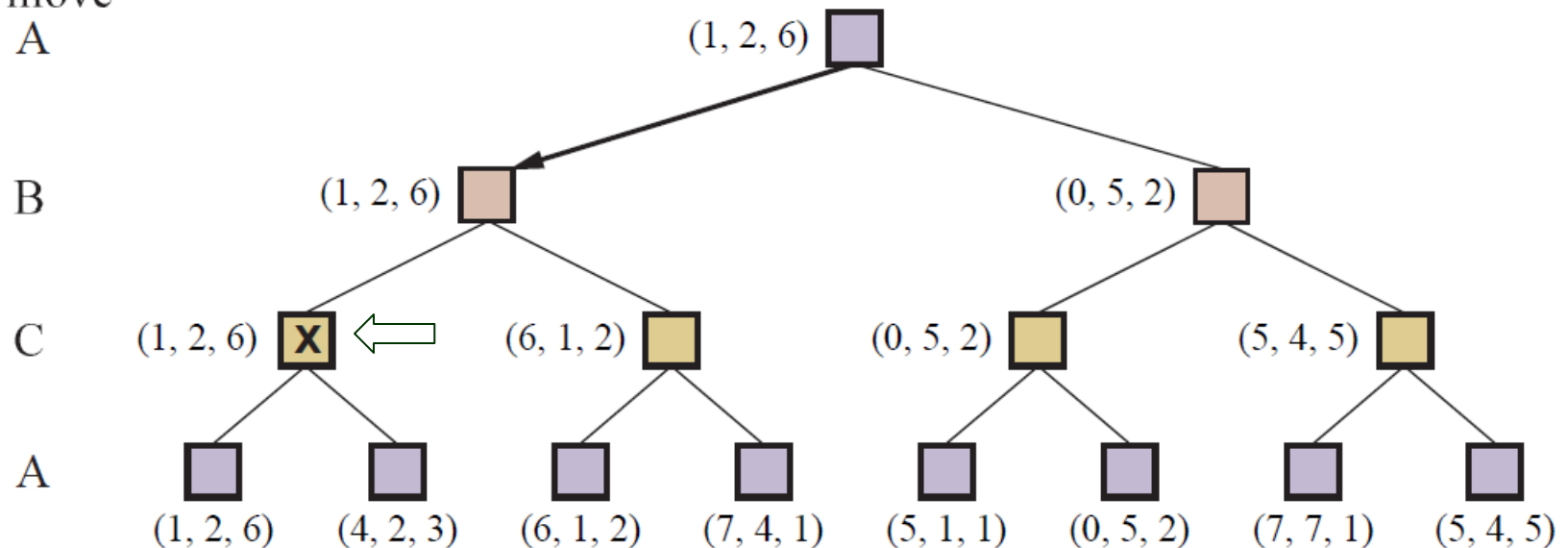
Multiplayer Games

Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C
Utility vector

to move
A



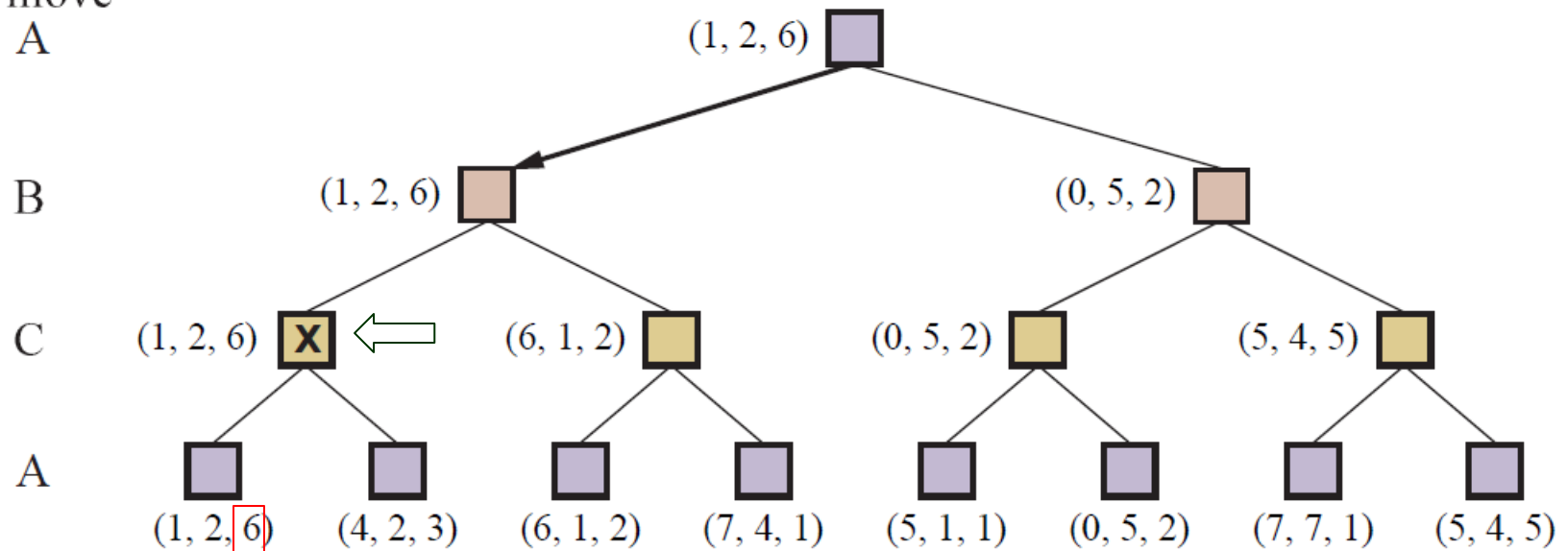
Multiplayer Games

Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C
Utility vector

to move
A



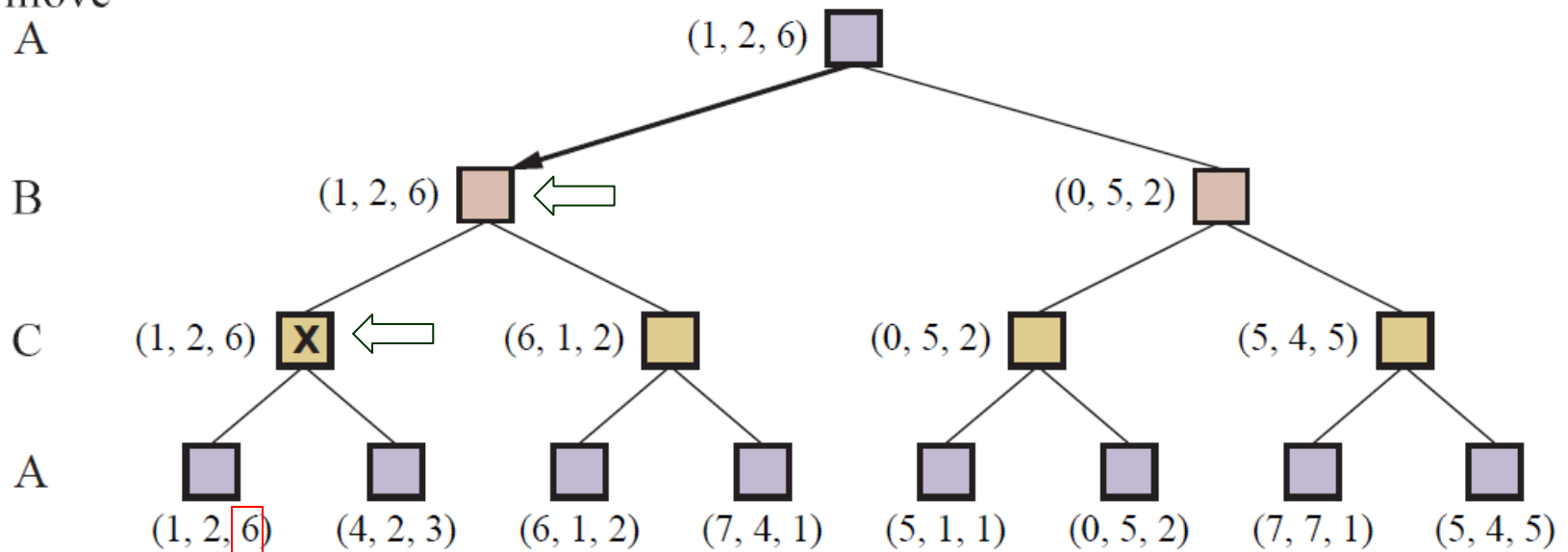
Multiplayer Games

Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C
 $\underbrace{\hspace{1.5cm}}$
Utility vector

to move
A



Multiplayer Games

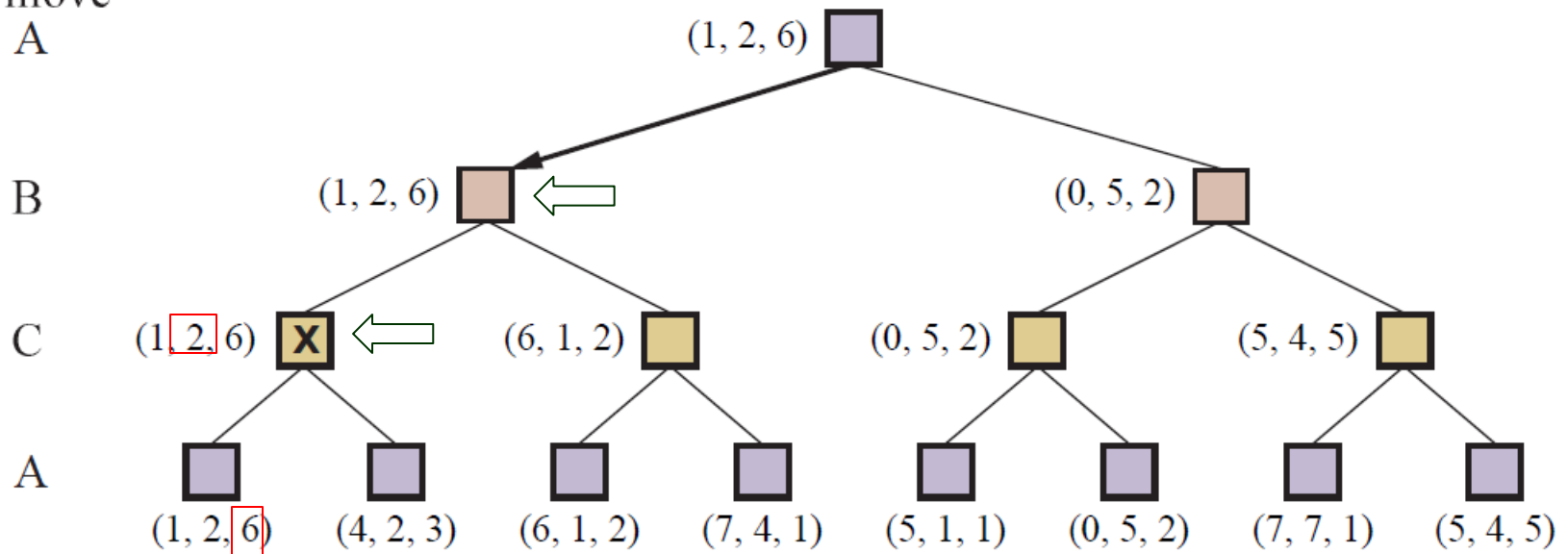
Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C

Utility vector

to move
A



Multiplayer Games

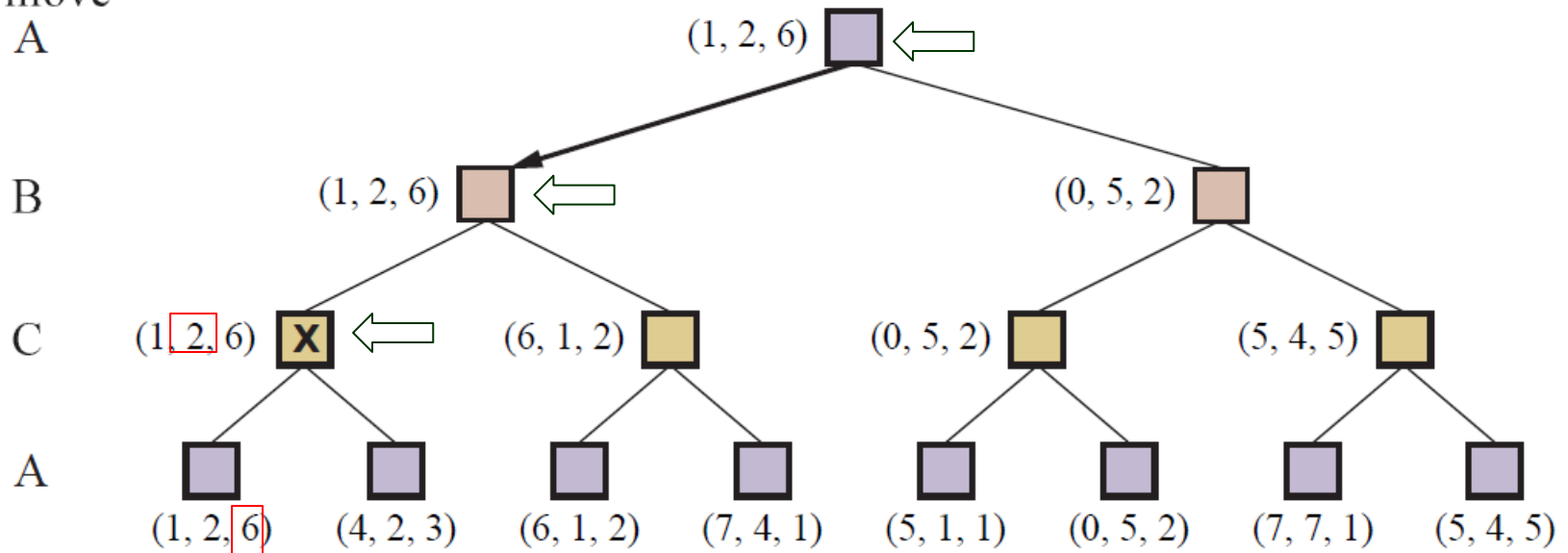
Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C

Utility vector

to move
A



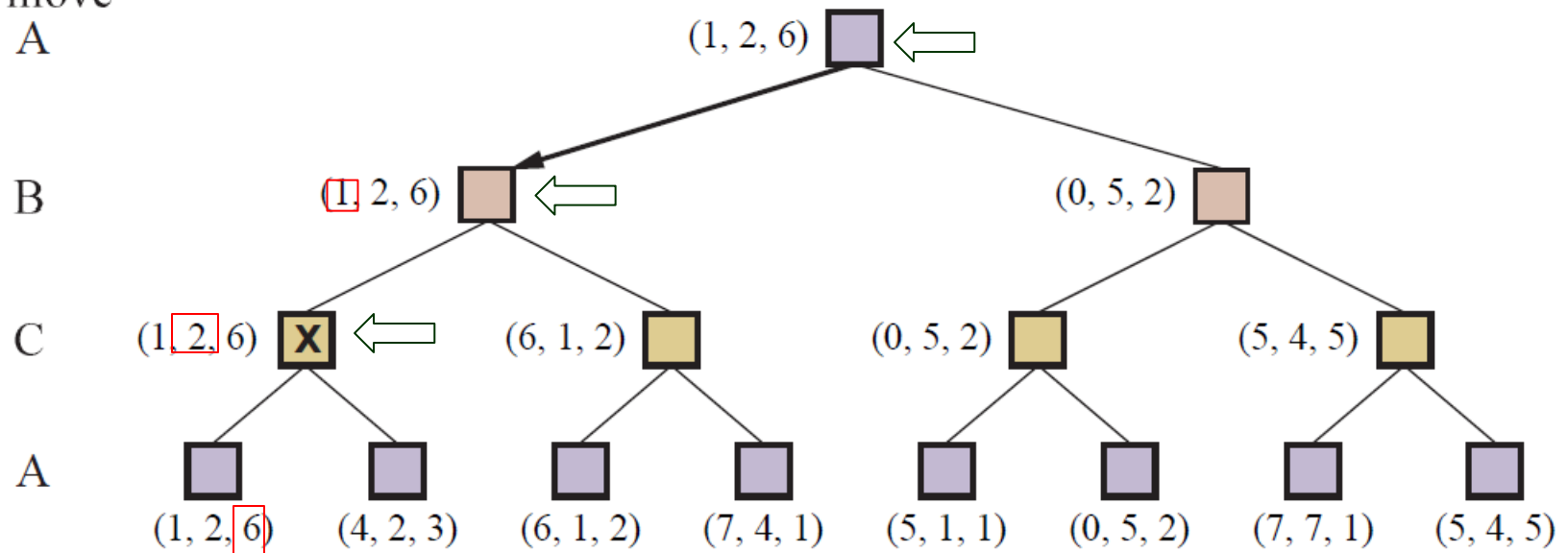
Multiplayer Games

Extend the minimax algorithm:

- Every node now has a *vector* of values.

$\langle v_A, v_B, v_C \rangle$ for three players A, B, C
 Utility vector

to move
A



Multiplayer Games

Extend the minimax algorithm:

- Every node now has a *vector* of values.

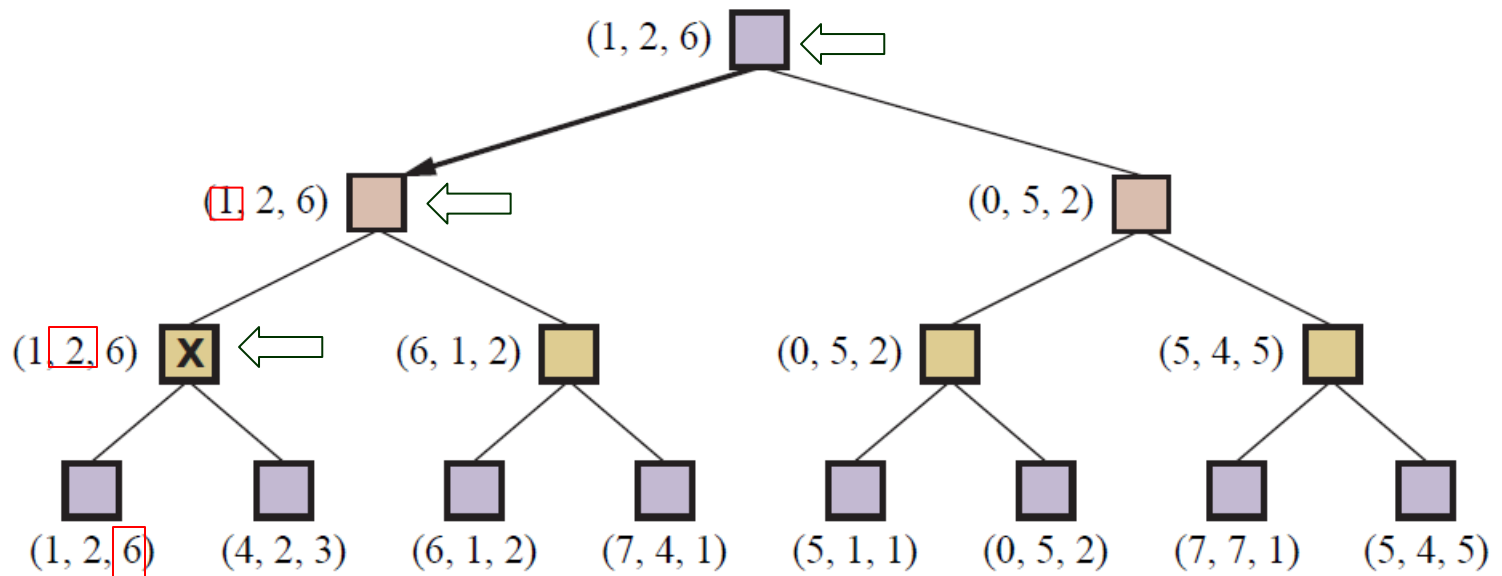
$\langle v_A, v_B, v_C \rangle$ for three players A, B, C
 Utility vector

to move
A

B

C

A



Backed-up value at a node n = utility vector of the successor state with the highest value for the player choosing at n